NO OTHER PERSONAL COMPUTER CAN COMPARE WITH SPECTRAVIDEO

SPECTRAVIDEO USERS GROUP (WELLINGTON) JULY 27, 1984
NEWSLETTER NUMBER 2

The next meeting is on Monday August 13th, Staff Common Room, Level D Wellington Clinical School, Main Street, Newtown at 7.30.

Peter Thakurdas will give about a 1/2 hour talk on machine code. This will assume absolutely no knowledge of machine code and will be based around the program on the last page of this newsletter.

There will be systems set up . bring along programs to demonstrate and/or swap.

The following meeting will be on September 17th

SECRETARY
Don Stanley
c/- Epidemiology Unit
Wellington Hospital
Wellington ph 855999x5654
or 5958

TREASURER
Colin Chin
P.O Box 2197
Wellington

Bits and Bytes magazine are interested in running a regular monthly article on MSX/SPECTRAVIDEO. Articles are in preparation for the next 3 months issues , but can be supplemented by other articles. If you have anything that you want to contribute (written articles or programs) , please contact Don Stanley for details.

MEMBERS .. welcome to the following new members -- Dennis Timmons,Tony Stanley,Tom Forster,Garth Thornton,Peter Thakurdas,J.W. Campin,Colin Horne,Garry Clark,J Roberts,Daniel Tobler,Brian Speirs,Alex Bridger,Michael Pollard

BASIC NOTES .. The Disk Basic Directory ..

This article aims to give some suggestions for writing programs which access data files on the disk. Two of the pitfalls with such programs are ..

1) user types in a non-existent file name
2) user types in an existing file name, but it is not an ASCII file

The first question is not too difficult to get round. Suppose the user types FILE.DST instead of FILE.DAT. If you incorporate NO error checking, the program will fail, so try using the ON ERROR statement like this ...

```
10 CLS:LOCATE 0,10:PRINT"FILE NAME"
20 LOCATE10,10:A$=INKEY$:IF A$="" GOTO20
30 IF ASC(A$)=13 GOTO60
40 IF ASC(A$)>=32 AND ASC(A$)<=126 THEN NF$=NF$+A$
50 LOCATE 10,10:PRINT NF$:GOTO20
60 ON ERROR GOTO5000
70 OPEN "1:"+NF$ FOR INPUT AS 1
80 ON ERROR GOTO0
 .
 .
4990 END
5000 PRINT "INVALID FILE NAME ...."
5010 FOR I=1 TO 500:NEXT
5020 RESUME10
```

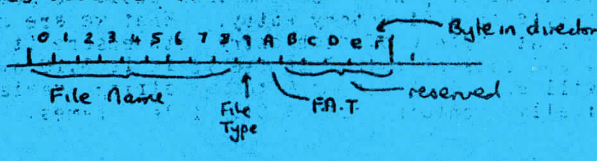
The crucial part of this program is the ON ERROR statements. The first at line 60 causes a branch to line 5000 if an error occurs. Since there is only 1 line interpreted while this error command is on (ie line 70), thus line 5000 can only be reached by an open file error at line 70, ie if the user enters an invalid file name. The second ON ERROR command causes normal error trapping to resume.

OK this solves problem one. But what do you do if your program has a valid file name entered, but it is a binary file. This will cause problems when your program expects an ASCII file as input.

To solve this problem we need to use a powerful command DSKI$. This statement causes an entire sector of the disk to be transferred to a program variable. In fact we will transfer chunks of the disks file directory to the program.

The SV disk directory is a portion of disk reserved for information about the files on it. There is provision for up to 208 files in the directory, however that doesn't mean you can have 208 files on the disk as it depends on the size of the files.

The directory has 16 bytes reserved for each file. These bytes are used as follows ...



Using DSKI$ causes an entire sector from a specified disk track to be transferred to a program variable. The directory makes use of the first 13 sectors on track 20. To assign a program variable the value of the first sector on track 20 use

```
A$=DSKI$(1,20,1)
```

(A$ will be 255 characters long, the last byte of the last file on the directory track is left off)

How does this help us? It is the byte at position 9 that we are interested in. The possible values of this byte are .. (values in decimal)

00 0 .. unprotected ASCII file (sequential or random access)
10 16 .. write protected ASCII file (sequential or random access)
&40 64 .. read after write check ASCII file (sequential or random file)

01 1 .. unprotected machine code file
11 17 .. write protected machine code file
41 65 .. read after write check machine code file

80 128 .. unprotected binary file
90 144 .. write protected binary file
C0 192 .. read after write check binary file

A0 160 .. unprotected screen save file (created with SAVE "filename",s)
B0 176 .. write protected screen save file
E0 224 .. read after write check screen save file

OK, we are getting near to a solution to problem 2. However before solving it completely lets look at 2 other of the bytes in the directory (refer previous diagram).

Byte 0 is the first character of the filename. But, if the file has been deleted this byte is set to zero. If there are no more directory entries then this byte is 255 (&hFF).

Byte 10 points to the file allocation table. This is a portion of the disk which keeps a map of where each file is on the disk. When a file is stored on the disk it is not necessarily stored as one contiguous chunk, so the file allocation table tells the DOS where to find the file.

Lets solve the second problem. There are 13 directory sectors. We only need to read them until we get an &hFF as the first byte of a directory entry. What we are going to do here is look at byte 0 of each entry (to ensure it is not zero or 255), and look at byte 9 to see if it is 0,16 or 64. If it is, we will append bytes 0-8 to a string variable which will eventually contain all ASCII file names. Here is the subroutine ..

```
1000 FOR S=1 TO 13
1010   A$=DSKI$(1,20+S)
1020   FOR I=10 TO LEN(A$) STEP 16
1030     IF ASC(MID$(A$,I-S,1))=&HFF GOTO1100
1040     CH=ASC(MID$(A$,I,1))
1050     IF CH<>64 AND CH<>16 AND CH<>0 GOTO1080
1060     IF ASC(MID$(A$,I-S,1))=0 GOTO1080
1070     VA$=VA$+STRING$(1,32)+MID$(A$,I-S,S)
1080 [illegible]
1090 NEXT
1100 RETURN
1200 REM THIS ROUTINE CHECKS AN INPUT FILE NAME TO SEE IF IT IS
1210 REM   IN THE STRING OF ASCII FILES
1220 CH=INSTR(VA$,NF$)
1230 IF CH=0 THEN PRINT "INVALID FILE NAME " ELSE GOTO1240
1235 FOR I=1 TO 300:NEXT:RETURNnnn
1240 OPEN "1:"+NF$ FOR INPUT AS 1
1250 RETURN
```

The RETURNnnn is just a line number to RETURN to to cope with the invalid file name having been entered. If the file name was valid the file is opened and processing goes back to the line after the line which called the subroutine.

There is still one problem , that no distinction is made between sequential and random files in the directory , but they behave differently if used as input. The simplest way to avoid this is not to mix sequential and random files on the same disk. Other disk errors can be trapped as in example one previously the whole point of this routine is to prevent accidental entry of a binary or machine code file to a program.

MORE CORRECTIONS TO MANUALS ... THE SV328 USERS GUIDE

PAGE	ERROR
14	It states that the contents of the SV328 package is an SV318 computer
	Misleading. It states that a line appears on the screen when it is really only a row of spaced-out boxes
129	~~Line 20 should read~~ CIRCLE (128,96),80,13,3.14,6.28
143	Line 180 should read STRIG(3)
154	The formulas are correct but they have not noted that some of these functions are not ~~defined for certain values of~~ X. All the

functions which involve SQR(-X*X + 1) are only defined when the absolute value of X is between plus or minus 1. All the formulae involving LOG((1+X)/(1-X)) are only defined when X lies between plus or minus 1. These are due to the mathematics of these functions, not to any fault with the functions . (ED)

158 ..... Error #26 is not FOR without NEXT

159 ..... Error #29 and #30 are non-existent as WHILE and WEND are not part of this BASIC

161 .. .. Some of the SV BASIC RESERVED WORDS do not exist, and others are not included

170 ..... line 50 should be SOUND 8,I

176 ..... Misleading ... your name does not appear at the top of the screen, instead 'YOUR NAME' appears.

Thanks to Elliott Young for locating these

AND ... THE SV318 BASIC QUICK REFERENCE GUIDE

10 ...... DEF FNx[(arg>,] .. note that the x in this refers to the number of the defined function and defaults at 1. The x in the example is NOT related to the x in the command.

12 ..... MID$ ... change with m number of character to with m characters substituted in the nth position .

12 ..... the example ON 1 GOSUB 100,200,300 should read ON I GOSUB 100,200,300

12 ..... substitute PORT for PROT in the OUT statement

13 ..... SWITCH returns a -1 if in bank 2 , a 0 if in bank 1

14 ..... CIRCLE .. the example has a syntax error there should be a radius immediately after the 96 . .

19 ...... ON ERROR GOSUB should be ON ERROR GOTO

25 ...... COBL(exp) should be CDBL(exp)

IF YOU FIND ANY MORE . PLEASE SEND THEM IN

ODDS AND ENDS

An error in some 80 column card manuals that will stop you switching back to 80 columns in CP/M is that the command STAT CON:=UR1: should be STAT CON:=UC1:

Despite what the manuals say , WHILE , WEND , COMMON and CHAIN are not available in DISK BASIC . Does anyone know any way to load a program from another program and retain the variables from the first program for use in the second (ie this is what COMMON and CHAIN usually do).

Bring your SV along to meetings.. come a bit earlier say 7-7.15 to set the system up ...

The Ascii character set includes a character ` ie CHR$(96), for which there is no obvious provision on the keyboard. This character is obtained by using right graph and the [key simultaneously . Alternatively use CNTL/RIGHT-GRAPH/Y simultaneously this sends a pair of characters to BASIC , the second of these is ascii 96.

The program ARTIST has a bug that when a picture is loaded for changing the cursor sprite disappears and you can't tell where on the screen you are. This is caused by the loading of a screen save file (which completely resets the video ram where the sprite definitions are stored). To correct this change the BASIC program as follows ..

Insert line 1135 RESTORE:S$=""

change line 5050 by replacing RETURN with GOSUB1700:RETURN150

This causes the cursor sprite to be reconstructed whenever a picture was loaded.

Does anyone know how to use the BASIC command MON ????????????????

Software prices .. if you think software is expensive here then here are some comparisons with Australian prices . Games and other standard tape software are each $29.95A which translates to about $40.00NZ compared with $18.00NZ here. But the cartridge games are $49.95A about the same (if not slightly less) than here.

CP/M software is more expensive than here. Take BASIC-80 compiler . $675.00A . DBASE2; $952.00A; COBOL-80 . $1195.00A as examples.

MORE REVIEWS .. Creative Computing . September 1983 pp16-26 Electronics Today International , October 1983 pp44-47

A couple of acknowledgements ... thanks to Einstein Scientific for sponsoring postage of newsletters to members and to Moonshine Computers for printing newsletters ....

We would like to include a short section in the newsletter each week on machine code and CP/M . If anyone would like to contribute to this (about a page of A4 including program chunks and explanations please ring the secretary.

Also lets have contributions for the newsletter , ANYTHING relating to the SV's or MSX is suitable

---

The next article should be of interest to all ... ensure that you have the ability to solder cleanly before trying this ...

PLEASE INSERT ARTICLE STARTING RESET

The following article has been contributed by Peter Thakurdas .

PLEASE INSERT ARTICLE ON USING THE SV328 AS A TERMINAL

PLEASE INSERT THE TWO PAGES ON CP/M, BRIEF NOTES ON CP/M AS FIRST AND THE MACHINE LANGUAGE PROGRAM AND NOTES FROM PETER THAKURDAS AS THE LAST PAGE

PLEASE INSERT ADVERTISEMENTS FOR MOONSHINE COMPUTERS AND FOR EINSTEIN COMPUTERS WHEREVER CONVENIENT FOR YOU

PLEASE INSERT MEMBERSHIP FORM NEAR BACK OF NEWSLETTER SOMEWHERE

ENQUIRE ABOUT OUR CLUB MEMBERS DISCOUNT.

MOONSHINE COMPUTERS & PERIPHERALS
== SPECTRAVIDEO ==

Effective from 15 June 1984.

Hardware Price List

Product	Retail:	Tax Exempt:	Product	Retail:	Tax Exempt
SV318 Computer *	$795 ~~$695.00~~	$546.07	SV328 Computer *	1095 ~~$995.00~~	$781.78
SV105 Graphics tablet	275 ~~$199.00~~	$156.35	SV101 Quickshot Joystick	22 ~~$ 22.95~~	$ 18.03
SV601 Super Expander	350 ~~$295.00~~	$231.78	SV602 Mini Expander	50 ~~$ 41.00~~	$ 32.21
SV603 Coleco Adaptor	165 ~~$139.00~~	$109.21	SV801 Disk Controller	295 ~~$275.00~~	$216.07
SV802 Centronics Interface	195 ~~$193.00~~	$151.64	SV803 16k Ram	95 ~~$ 95.00~~	$ 74.64
SV805 RS-232 Interface	195 ~~$193.00~~	$151.64	SV806 80 Column Card	295 ~~$295.00~~	$231.78
SV807 64K RAM	295 ~~$275.00~~	$216.07	SV902 Disk Drive	895 ~~$795.00~~	$624.64
~~SV903 Cassette Drive~~	~~105 $129.00~~	~~$101.35~~	SV904 Cassette Drive	105 ~~$ 89.95~~	~~$ 62.30~~
SVPAR Parallel Printer Cable	55 ~~$ 45.00~~	$ 45.00			

Programs for $18.00

SD302T 3-D Tic-Tac-Toe	SD304T Acutype	SD233TJ Armoured Assault	SD901T Artist
SD301T Biorhythm	SD305T Bos	SD307T Car Ace	SD308T Compatibility
SD908T Educational 3-in-1	SD917T Family Collection	SD268T Financial Calculator	SD906T German Language Program
SD904T Gobbler	SD916T Golf	SD910T Ghostrap	SD903T Graveyard
SD902T Hangman	SD229T Home Economist	SD310T Horse Race	SD900T Household Inventory
SD235T Intro to Basic	SD306T Juno Lander	SD293TE Kung Fu Master	SD914T Labyrinth
SD905T Maths	SD918T Microstics 1	SD292TE Ninja	SD255T Nomis
SD303T Number Game	SD909T N.Z.S.I	SD211TE Old Mac Farmer	SD912TE Perilous Journey
SD243TE Sasa	SD269T Spectra Address Book	SD309T Spectra Break	SD227T Spectra Checkbook
SD220T Spectra Diary	SD275T Spectra File Cabinet	SD267T Spectra Font Editor	SD915T Spectra Mind
SD266T Spectra Sprite Editor	SD911T Spectra Trader	SD288T Spectra Type	SD234T Spectron
SD256T Star Word	SD247TE Telebunny	SD216TE Tetra Horror	SD913T Treasure Hunt
SD242TE Turboat			

+ ON CASSETTE (DISC EXTRA $10)

Other Programs

SDSMAT BUSINESS PROGRAMS $ 240.00 — SD907T Textprint 33 ~~$25.00~~ + A/C ANALYSIS 55.00

+DEBTORS LEDGER 50.00
+CLUB RECORDS 25.00

Cartridges

SD291C Flipper Slipper	~~$59.95~~	SD232C Frantic Freddy	69.95 ~~$59.95~~	SD294C Just Write Jr	$79.95
SD236C Music Mentor	69.95 ~~$59.95~~	SD220C Sector Alpha	$129.00	SD234C Spectron	69.95 ~~$59.95~~
SD237C Super Cross Force	~~$59.95~~				

Note: C = Cartridge.
T = Cassette.
TE = Cassette. Requires SV328 or SV318 plus SV803 16K RAM.
TJ = Cassette. Requires SV101 Joystick.

NEW HARDWARE
GRAPHICS TABLET $295
SV605 EXPANSION WITH 1 BUILT IN DRIVE $1795 (INC CONTROLLER & CENTRO)
SV605 WITH 2 DRIVES 2250

AND NOW A LARGE RANGE OF "CP/M" SOFTWARE FOR SV AVAILABLE TO ORDER

WELLINGTON:
Shop 13, Mc Donalds Centre,
Herbert Street, City
Tel. 739-146

LOWER HUTT:
70 Victoria Street,
(Next to Hire Pool)
Tel. 697-757

* BOTH ARE FAMILY PACKS + PRICE INCLUDES 4 PROGRAMS + SV904 DATACASSETTE

EINSTEIN SCIENTIFIC
EINSTEIN SCIENTIFIC
EINSTEIN SCIENTIFIC

CP/M SOFTWARE ... Einsteins have a large range of CP/M software for *SPECTRAVIDEO* including Basic and Pascal Compilers , Wordstar Peachtext and many others ....

Also some hardware is still available at ***PRE-DEVALUATION*** prices ... especially plug in cards RS-232 , etc

EINSTEIN SCIENTIFIC
EINSTEIN SCIENTIFIC
EINSTEIN SCIENTIFIC

177 Willis Street , Wellington — for SpectraVideo Hardware ,Software and peripherals ...

call Derek Baker , Raju Bariani , Michael Pollard , 851055 for more information

USING THE SV328 AS A TERMINAL

I own a SV328 1-disk-drive system (with an RS-232 Interface) connected to a colour TV. Below is an outline of what was needed to get the SV328 to act like a terminal.

VT-52 Terminal Type & Host Computer

The CP/M Users Guide on page 7 mentions that the
"..Spectravideo Personal Computer emulates the popular video terminal known as the DEC VT-52 (Trademark of Digital Equipment Corp)."
Any computer which supports a VT-52 terminal (or similar) device could then be serious candidate as "host computer" for the SV328. The "VT-52" being a DEC trademark suggests that many of DEC's computers would be suitable.
The host for my SV328 is a DEC VAX-11/780 - the Wellington VAX of DSIR's computer network.

The RS-232 Connection

After establishing the host computer, the SV328 had to be connected to it. This was done through an RS232 interface (SV805), which sits on one of the slots in the Expander (SV601). The connections from the plug of the RS232 to a computer port socket need to be made with care, and would be best done by an appropriately qualified technician.

Terminal Program

Next a terminal program was needed for the SV328.
A terminal program basically acts as a "go-between" the two computers. It displays characters received from (and sends what you type to) the host computer through the RS232 interface. Technical details such as which ports (in the SV328) correspond to the RS232 can be found in the RS232 manual, that comes with the Interface.
And the terminal program runs on the SV328 so it may also initialise (or set up) the RS232 to the appropriate data transfer (baud) rate.

Apart from the 40 column display, I'm quite satisfied with its performance. (My RS232 is set to 1200 baud rate).
The terminal program has gone through various stages of refinement and growth. It is now also possible to transfer text files to (and from) the SV328 and use the SV328 as a graphics terminal.

I'm greatly indebted to fellow staff at the DSIR, who helped me do all this.

I would welcome any questions/comments/suggestions from others who may be trying to do similar things. My contact address is:

Peter Thakurdas, C/- AMD DSIR, PO Box 1335, Wellington.

Brief Notes on CP/M

As many of you will know, CP/M is an Operating System.
At the command level, it basically lets you type or list or delete files on disks and runs machine code programs.

Looking at CP/M commands, you'll see that there are 2 sorts:

1. Internal (or system) are "built-in" commands.
   For example : DIR or TYPE or SAVE (useful).
2. External commands are really machine code program names (without ".COM" ending.) For example : DUMP, STAT or LOAD.
   To get a list of these type: dir *.com

You can add to these "external" commands by writing your own machine code programs.

For large programs you could take the following steps:

1. Use ED, the line editor provided, to obtain a "source" file.
   This file contains machine code instructions written in ASSEMBLY LANGUAGE and has the ".ASM" filename ending.
   Eg prog1.asm

2. Use ASM, the assembler, to convert the symbolic language of Assembly Language to hexadecimal text, which is the readable character representation of machine code.
   ASM produces a file with ".HEX" ending. Eg prog1.hex
   It may (optional) also produce a ".PRN" file, which gives details of what was assembled. Eg prog1.prn

3. Use LOAD, the loader, to produce the final executable (and unreadable) ".COM" file. Eg. prog1.com

For small, short programs you could simply:

Use DDT to assemble the program as you type it in, quit using CTRL-C and save it as prog1.com file on disk.

The following briefly outlines what happens when you type a command for which there should be a corresponding ".COM" file present on disk.

For example: A>prog1 and-something-else

- If prog1.com is not present on disk, then you'll get the response: PROG1?

- If prog1.com is present on disk, then it's contents are loaded into memory starting at location 0100H.
  The 'and-something-else' is converted to uppercase ('AND-SOMETHING-ELSE') and loaded into memory starting at location 0081H. (This allows a way of passing parameters to a program (or command).)
  And execution of machine code instructions (starting at 0100H) begins. (ie The command or program is run).

Here is how to get a colour changing command in CP/M.

Type whats under "8080 version" after CP/M boot up.
The Z80 version is supplied simply for comparison.
All values are in hexadecimal. (ie 3a really means 3aH = 3*16 + 10 = 58)

```
 8080 version                        Z80 version        Comments
 ------------                        -----------        --------
DDT
a100                          Address
lda 0082                         0100 ld  a,(0082)   ; Load byte at 0082 to 'a' reg
sui 30                                sub 30         ; Subtract 30
cpi 09                                cp  09         ; Is it less than 09?
jc  010c                              jp  c,010c     ; Yes, jump to 010c
sui 07                                sub 07         ; (No) subtract 07
ral                              010c rla            ; Rotate left 'a' reg
ral                                   rla            ;   thru carry flag
ral                                   rla            ;   four times
ral                                   rla            ;
ani f0                                and f0         ; Mask out lower 4 bits
mov b,a                               ld  b,a        ; Store 'a' reg in 'b' reg
lda 0083                              ld  a,(0083)   ; Load byte at 0083 to 'a' reg
sui 30                                sub 30         ; Subtract 30
cpi 09                                cp  09         ; Is it less than 09?
jc  011f                              jp  c,011f     ; Yes, jump to 011f
sui 07                                sub 07         ; (No) subtract 07
ani 0f                           011f and 0f         ; Mask out higher 4 bits
ora b                                 or  b          ; OR 'a' and 'b' regs
out 81                                out (81),a     ; Send result to VDP as data
mvi a,87                              ld  a,87       ; Load 'a' with 87
out 81                                out (81),a     ; Send 'a' (87) to VDP as com
ret                              0128 ret            ; RETURN to CP/M
```

CTRL-C (to get to "A>" prompt)
save 1 colour.com

The save command here saves 1 "page" of memory, from 0100 to 01ff, to disk (to a file called "colour.com").
("save 2 colour.com" would have saved 2 pages, ie. from 0100 to 02ff, to colour.com. But saving 2 pages is not necessary since colour.com fits in 1 page.)

Now experiment, type :

```
colour 13    (equivalent to "color 1,3" in BASIC)
colour f4    (   "          "  "color 15,4" or "color &hf,&h4" in BASIC)
 etc...
```

Note: The program down to the first "out" command converts the two-letter hexidecimal character string (eg 13 or f4), which uses 2 bytes (at 0082 and 0083 in memory), to a 1 byte hexidecimal value in the 'a' register.
Then the "out" command sends this value as data to the Visual Display Processor (VDP) chip. That is followed by another "out" command which tells the VDP that the data previously sent was for specifying text and background colour.

Spectra-Video Users Group (WGTN)

MEMBERSHIP FORM

NAME ______________________________

ADDRESS ______________________________

____

____

PHONE ______________________________

INTERESTS Basic __ Assembler __ Games __ CP/M __ Education __

ATTENDANCE AT MEETINGS

Usually __ Sometimes __ Occassionally __ Newsletters Only __

------------ D O N O T F I L L I N H E R E ------------

PAID DATE EXPIRES SIGNED

SEND COMPLETED FORM + $10.00 ANNUAL SUBSCRIPTION TO

Spectra-Video Users Group (wgtn)
c/- Don Stanley
Epidemiology Unit or P.O. Box 7057 or P.O. Box 2187
Wellington Hospital Wellington Wellington

Reset

You have just finished writing a marvelous new program (BASIC or assembly), have been prudent and written it to cassette or disk, and now it is time to try it out.
You press RUN,..... and nothing happens. Your machine has gone for a walk through memory, leaving you and your program far behind. There is nothing for it but to use your ultimate weapon against program failure, THE POWER SWITCH.

Or is there?

Every time you turn your computer on or off, the components in the power supply are subjected to spikes and voltages at their operating maximum. The fewer times you do this, the longer your computer will last.

The microcomputer chip in the Spectravideo (the Z80) has one pin specifically allocated to resetting the system without touching the power supply. In fact when you turn the power on, the power supply has a special circuit that uses this pin to reset the computer when the power has stabilised. Other computers have a switch across this pin that the user can use to reset the system when he(or she) want, and with 15 minutes work and $2.50 worth of parts, you can to.

For this job, you will need a fine tip soldering iron, some fine resin core solder, about 30cm of hookup wire, a small momentary action push button switch, and a 10 uF (microFarad) tantalum capacitor.

Disconnect your computer from all the other parts of the system (especially the power) and turn it on its back, with feet in the air. Seven Philips head screws hold the case together (on the 328); remove them all. Holding the case together, turn it back on its feet again. Turn the case around so that you are looking at the expansion and cassette slots. Lift the top half of the case off by pivoting it on the front (keyboard) edge of he bottom half. The keyboard is connected to the main board by two flexible (but fragile) connectors and it is best to leave these connected. Lay the top half of the case on the table, key side down. There, you have now exposed the innards of your Spectravideo to the world and VOIDED YOUR WARRANTY.

About five centimetres up from the back of the case is a large black integrated circuit. If you twist your head around you will see that it it labelled (on the circuit board) IC37 and printed on the beastie itself is Z80. Refer to the map.

Just above the Z80 chip the board has the characters 328-01 printed on it.Just above,or below depending on whether your head is still twisted, is a round golden spot. This spot is a plated through hole, and is where one of your wire ends will go. Strip the insulation from about four millimetres at one end of your wire, and tin the end with solder. Place the tinned end in the above fore-mentioned hole and apply a little solder and heat.

While that connection is cooling, look to the right of the Z80 chip and you will see a resister labelled R16. Just down and to the left of this resister are two plated holes, and the one we want is the leftmost one. Find the middle point in your wire and cut. Take the bit that is hanging free now and strip about four millimetres of insulation off. Tin. Place this tinned end in the hole previously identified and solder. You should now have two bits of wire sticking out of your circuit board.

A good place to fit the reset button is the top left hand side of the keyboard, as seen from the operator, sticking out the side. The button I chose does not protrude beyond the side of the case, makeing it very hard to bump accidently. I made a hole in the side of the bottom half of the case, about 10 millimetres in front of the screw pillar next to the video socket.

Check that your button can be installed after you have connected the wires to it, then strip back the insulation from both pieces of wire and solder them to the switch.

The next stage is the capacitor. This will be soldered across the switch, where the wires are soldered on now. Identify the positive leg of the capacitor (usually the one with the plus signs on it). This leg gets solderedto the switch terminal that is connected to the wire that is connected to the hole near the 328-01 (which is connected to the knee bone connected to the thigh bone...)

Now is the time to check all your connections for dry joints or solder bridges.

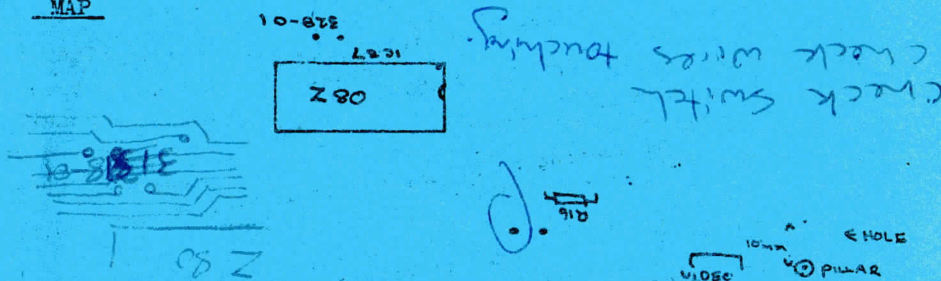
If it all looks ok, fit the switch into the hole in the case, and fit the top of the case, and finally screw the case back together.

To test this modification, connect your computer to the screen and the power, but not expander or cassette or anything else. Turn the power on, and if you get anything but the normal screen display, or no display at all, TURN OFF IMMEDIATELY. Open the case and go over all your connections, and the way thecapacitor is instal

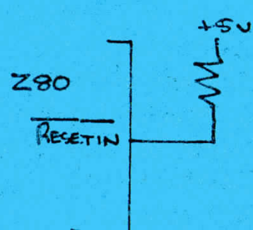
If the machine did start up normally, after the BASIC sign on message is displayed, press the new button once, briefly. The computer should go back into the start up self test display. If it doesn't, open the case and check.

If all is working well, turn your computer off and reconnect all your extras, and you are away with a new facility.
Good Luck.. Ross Wakelin

MAP



Circuit before



Circuit after